RDD: Resilient Distributed Datasets

Contents

- Transformaciones y acciones sobre RDDs

Los RDD constituyen la **abstracción fundamental de Spark** (más detalles en http://spark.apache.org/docs/latest/rdd-programming-guide.html).

> **Note**
>
> Un RDD es una colección no ordenada (bag) de objetos distribuida en la memoria RAM de los nodos del cluster.

La colección está dividida en *particiones*, y cada una está en la memoria RAM de un nodo distinto del clúster. Desgranando el nombre, tenemos que

- Resilient (resistente, adaptable): es posible reconstruir un RDD que estaba en memoria a pesar de que una de las máquinas falle, gracias al DAG de ejecución (Directed Acyclic Graph, el grafo de ejecución). Comentaremos esto en detalle más adelante.
- Distributed (distribuido): los objetos de la colección están divididos en *particiones* que están distribuidas en la memoria principal de los nodos del cluster. La colección no está ordenada, por lo que no se puede acceder mediante una posición a objetos individuales.
- Dataset: la colección representa un conjunto de datos que estamos procesando de forma paralela y distribuida, transformándola, calculando agregaciones, etc.

La Fig. 4 representa tres RDDs diferentes distribuidos en la memoria RAM de un cluster de 4 nodos. No todos los RDD tienen el mismo número de particiones. En la figura, uno de los RDD tiene solo 2 particiones, otro tiene 3 y otro tiene 4. La idea es similar al almacenamiento de los ficheros en HDFS, donde están distribuidos entre los discos duros de los nodos, con la diferencia de que en este caso los RDD están distribuidos en la memoria RAM de los nodos, y además, **no hay replicación de cada partición**. Si un nodo falla, es posible reconstruir las

Skip to main content

DAG, que mantiene la traza de cómo se construyeron. El DAG es otro mecanismo que proporciona robustez sin necesidad de replicar las particiones de un RDD. Nótese también que, a pesar de que la Fig. 4 muestra una sola partición de cada RDD en cada nodo, lo habitual es que en la memoria de un mismo nodo haya numerosas particiones de un mismo RDD (de hecho es normal que los RDD que se van calculando tengan decenas o cientos de particiones).

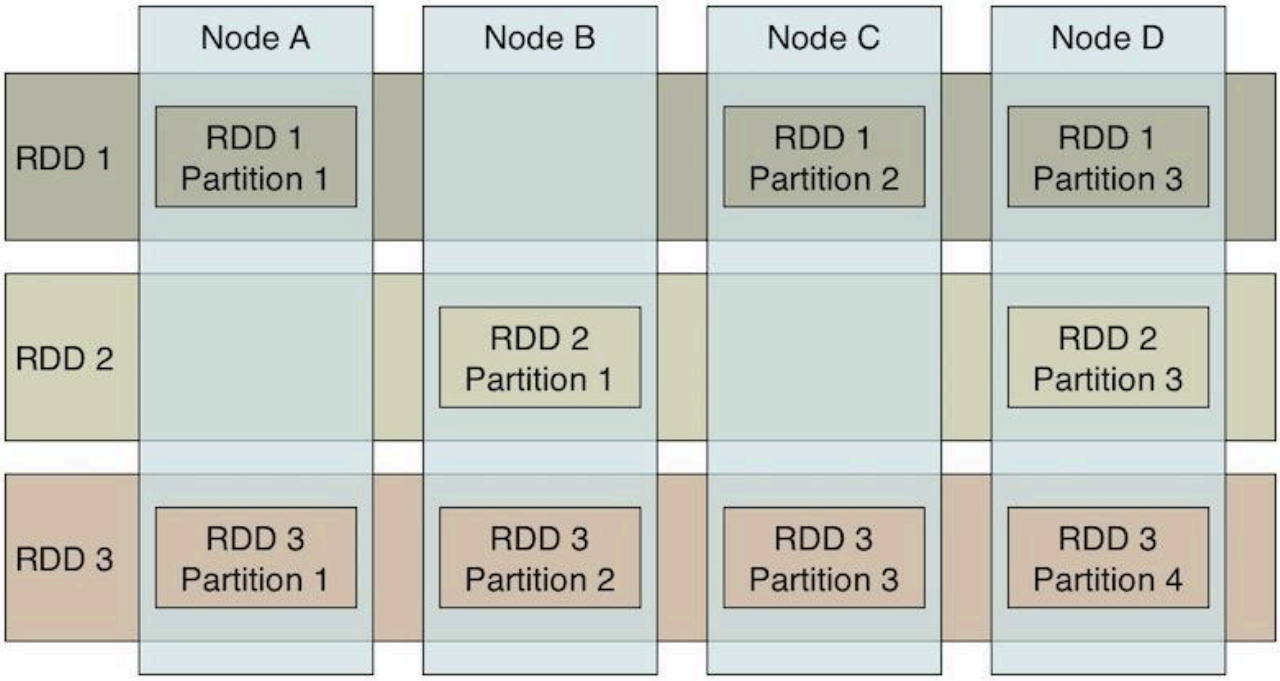


Fig. 4. Representación de tres RDDs en un cluster de Spark con cuatro workers.

Inmutabilidad de un RDD: el contenido de un RDD no puede modificarse una vez creado. Lo que hacemos es aplicar transformaciones a los RDD para obtener otros nuevos, pero los datos del RDD existente no se alteran.

La idea de la ejecución es que, cuando aplicamos una transformación, se ejecuta en paralelo sobre todas las particiones del RDD, de manera transparente al programador, para dar lugar a un nuevo RDD como resultado, cuyas particiones son el resultado de aplicar la transformación sobre cada una de las particiones del original.

- Ejemplo: dado un RDD de números reales, para multiplicar cada elemento por 2, aplicamos una transformación que actúa en cada elemento y lo multiplica por 2. Spark *lleva nuestro código de la transformación* (lo serializa y lo envía por la red) a cada uno de los nodos del cluster donde haya particiones de ese RDD, y lo ejecuta en ese nodo para que actúe en cada elemento de esa partición. Todo de manera transparente al programador.
    - Una vez más, *los datos son el centro, lo más importante; no se mueven salvo que sea*

Skip to main content

datos pero otros sí, como veremos después.

- *Partición*: subconjunto de los objetos de un RDD que están presentes en un mismo nodo. **Es la unidad de datos mínima sobre la que se ejecuta una tarea de transformación de manera independiente al resto de particiones**. Idealmente, tendría que haber al menos tantas particiones como cores físicos (procesadores) disponibles en nuestro cluster. De esta manera nos aseguramos de que todos los cores estarán ocupados incluso en el caso de que todos estuviesen libres y nadie más que nuestra aplicación estuviese ocupando ese cluster.

Originalmente, los programadores de Spark trabajaban a nivel de RDD. En Spark 1.6 se introdujeron los DataFrames, que definiremos más adelante, y desde Spark 2.0 los propios creadores *recomiendan encarecidamente no utilizar los RDD sino siempre DataFrames con SparkSQL* como veremos en secciones posteriores. Por ese motivo, [todos los ejemplos que veremos en clase utilizan DataFrames desde el lenguaje Python, importando la biblioteca *pyspark*.]{.underline} En el anexo de este tema puedes encontrar un código de ejemplo que utiliza RDDs y un resumen de las operaciones más frecuentes con RDDs. No obstante, y puesto que un DataFrame envuelve dentro a un RDD facilitando su manejo, **todos los conceptos sobre transformaciones y acciones que se tratan en la sección siguiente son válidos cuando manejamos un DataFrame.**

Transformaciones y acciones sobre RDDs

Existen dos tipos de operaciones que podemos llevar a cabo cuando usamos la API de Spark:

- **Transformación**: operación que se ejecuta sobre un RDD y devuelve un nuevo RDD, en el que sus elementos se han modificado de algún modo. Son *lazy* (perezosas): no se ejecuta nada hasta que Spark encuentra una acción. Mientras tanto, Spark simplemente añade la transformación al grafo de ejecución (el **DAG**, *Directed Acyclic Graph*) que mantiene la trazabilidad y permite la resiliency.
  - El DAG tiene anotada la secuencia de transformaciones que se realizaron para obtener cada RDD concreto que se va creando en nuestro código.
- Si la transformación no implica shuffle (movimiento de datos entre nodos) se denomina *narrow*, y cada partición da lugar a otra en el mismo nodo. En caso contrario, cuando no es posible completarla en cada nodo sin acceder a los datos que hay en otros nodos, se denomina *wide*.
- **Acción**: operación que se aplica sobre un RDD y calcula un resultado (generalmente un tipo simple del lenguaje de programación, como enteros, doubles, listas, etc) que es enviado al driver (el cual corre en una máquina)

Skip to main content

- IMPORTANTE: el resultado de la acción debe caber en la memoria de la máquina donde se está ejecutando el proceso driver.
- Una acción desencadena instantáneamente el cálculo de toda la secuencia de transformaciones intermedias, y la materialización de los RDDs involucrados.
- Una vez materializado un RDD, se aplica la transformación que toque según indica el DAG para generar el siguiente RDD, y el anterior se libera (no permanece en la memoria RAM, salvo que se indique expresamente mediante el método cache()). Un RDD cacheado permanece materializado en la RAM y no es necesario recalcularlo después de que se haya materializado la primera vez cuando esté involucrado en una secuencia de transformaciones.

Por defecto, el punto de partida del DAG son operaciones de lectura de datos desde una fuente de datos como por ejemplo HDFS, o bien el sistema Amazon S3, o alguna base de datos (distribuida o no) o similar. Si ningún RDD intermedio ha sido cacheado, cualquier operación que haga referencia a un RDD exigirá reconstruir toda la secuencia de transformaciones previas a dicho RDD, empezando en la lectura de los datos, excepto que alguno de los RDD intermedios haya sido cacheado. En ese caso, las operaciones empiezan desde el RDD cacheado y no es necesario remontarse al origen de datos.

Existen ciertas operaciones de la API de Spark que **no son transformaciones ni acciones**, como por ejemplo *cache()*, sino que sirven para configurar, habilitar o deshabilitar ciertos comportamientos, o para obtener características relativas a la distribución física de un RDD (consultar el número de particiones que tiene, consultar si está cacheado, consultar el esquema –nombres y tipos de las columnas– de un DataFrame, etc). Dichas operaciones se ejecutan en el driver y no requieren intervención de las máquinas del cluster de Spark.